ORGNZ-09 LAB: Enterprise Data Organization Patterns - Hands-on Exercises

> **Series:** ORGNZ | **Notebook:** 9 of 10 | **Type:** LAB | **Created:** February 2026 | **Last Updated:** 02/19/2026

Overview

This lab notebook contains 3 hands-on exercises extracted from **ORGNZ-09: Enterprise Data Organization Patterns**. Complete the lecture notebook first, then work through these exercises to reinforce the concepts with real DQL queries against your Dynatrace environment.

Table of Contents

Prerequisites

Requirement	Details
Completed	ORGNZ-09: Enterprise Data Organization Patterns (lecture notebook)
Dynatrace Environment	SaaS tenant with Grail enabled
Permissions	`logs.read`, `metrics.read`, `entities.read`, `spans.read`

Exercise 1: Complete Architecture Example

ORGNZ-09: Enterprise Data Organization Patterns

> **Series:** ORGNZ | **Notebook:** 9 of 10 | **Created:** January 2026 | **Last Updated:** 02/09/2026

This notebook brings together buckets, segments, and security context into comprehensive enterprise patterns. Learn how to combine these mechanisms for effective data governance at scale.

Requirement	Details

---

1. Combining All Three Mechanisms
2. Enterprise Pattern 1: Line of Business Isolation
3. Enterprise Pattern 2: Environment-Based Tiering
4. Enterprise Pattern 3: Multi-Cloud Organization
5. Enterprise Pattern 4: Kubernetes-Native Organization

6. Decision Framework
7. Implementation Checklist
8. Auditing Your Organization
9. Best Practices Summary
10. Series Summary

----------|----------|
Dynatrace Account	Account-level administrative access
Permissions	Full IAM and bucket management permissions
Knowledge	Completed ORGNZ-01 through ORGNZ-08

By the end of this notebook, you will:
- Combine buckets, segments, and security context effectively
- Design enterprise-scale data organization architectures
- Implement common organizational patterns
- Plan data governance strategies

For comprehensive data governance, use all three mechanisms:

Mechanism	Purpose	When to Use
Buckets	Physical storage, retention, cost	Different retention needs, cost attribution
Segments	Logical filtering, views	Team-focused data views, cross-signal filtering
Security Context	Access enforcement	Fine-grained security requirements

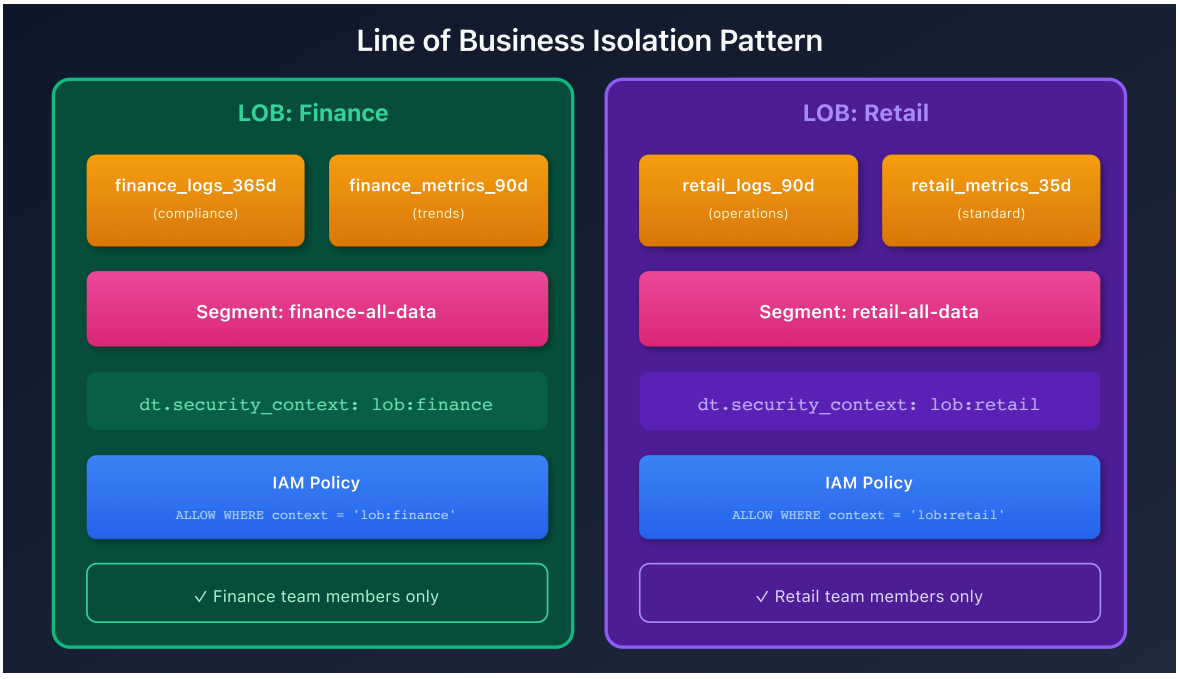
```
Finance Team Requirements:

Bucket: finance_logs_365d
  - 1 year retention for compliance
  - Cost attributed to Finance

Segment: finance-team-view
  - Filtered view of finance data
  - Includes related services and hosts

Security Context: lob:finance
  - IAM policy: ALLOW WHERE security_context = 'lob:finance'
  - Only finance team members can access
```

Complete isolation by business unit:



Step 1: Create buckets

```
finance_logs_365d (logs, 365 days)
finance_metrics_90d (metrics, 90 days)
retail_logs_90d (logs, 90 days)
```

Step 2: Configure OpenPipeline routing

```
processors:
  - type: route
    rules:
      - condition: "host.group starts-with 'finance-'"
        destination: "finance_logs_365d"
      - condition: "host.group starts-with 'retail-'"
        destination: "retail_logs_90d"
```

Step 3: Set security context

```
processors:
  - type: security-context
    rules:
      - condition: "host.group starts-with 'finance-'"
        context: "lob:finance"
      - condition: "host.group starts-with 'retail-'"
        context: "lob:retail"
```

Step 4: Create IAM policies

```
// Finance policy
ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH "finance_";
ALLOW storage:logs:read, storage:metrics:read WHERE
storage:dt.security_context = "lob:finance";
```

Different access levels for production vs non-production:

```
Production:
├── Buckets: prod_logs, prod_metrics, prod_spans
├── Security Context: env:production
├── Policy: ALLOW WHERE storage:dt.security_context = 'env:production'
├── Access: Restricted to production support team
└── Audit: Full audit logging enabled

Non-Production:
├── Buckets: default_logs, default_metrics, default_spans
├── Security Context: env:dev, env:staging, env:qa
├── Policy: ALLOW WHERE storage:dt.security_context MATCH ('env:dev*',
'env:staging*')
├── Access: Broader developer access
└── Audit: Reduced audit requirements
```

Group	Production Access	Non-Prod Access
SRE Team	Full	Full
Production Support	Read-only	None
Developers	None	Full
QA	None	Read-only

Organize by cloud provider and account:

```
AWS:
├── Bucket: aws_logs_35d
├── Security Context: cloud:aws/&lt;account-id&gt;
└── Policy: ALLOW WHERE storage:aws.account.id = '123456789012'

Azure:
├── Bucket: azure_logs_35d
├── Security Context: cloud:azure/&lt;subscription&gt;
└── Policy: ALLOW WHERE storage:azure.subscription = 'sub-id'

GCP:
├── Bucket: gcp_logs_35d
├── Security Context: cloud:gcp/&lt;project&gt;
└── Policy: ALLOW WHERE storage:gcp.project.id = 'my-project'
```

Leverage Kubernetes constructs for organization:

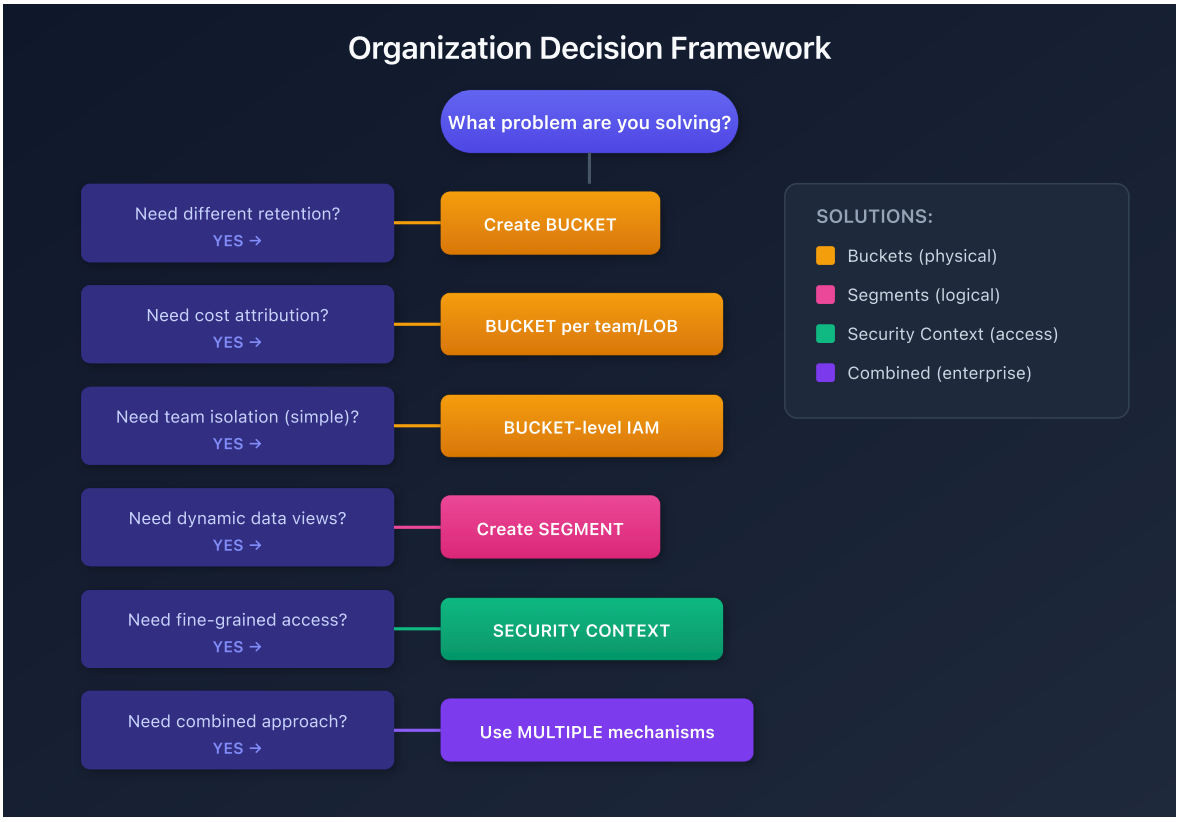
```
Cluster: main-cluster
├── Namespace: production
|   ├── Security Context: k8s:main-cluster/production
|   └── Policy: ALLOW WHERE storage:k8s.namespace.name = 'production'
|
├── Namespace: staging
|   ├── Security Context: k8s:main-cluster/staging
|   └── Policy: ALLOW WHERE storage:k8s.namespace.name = 'staging'
|
└── Namespace: development
    ├── Security Context: k8s:main-cluster/development
    └── Policy: ALLOW WHERE storage:k8s.namespace.name = 'development'
```

```
name: checkout-application
displayName: "Checkout App (All Environments)"

includes:
  - type: logs
    filter: "k8s.deployment.name == 'checkout'"

  - type: spans
    filter: "service.name == 'checkout'"
```

Use this framework to choose your organization strategy:



- [ ] Identified retention requirements
- [ ] Mapped organizational structure (teams, LOBs, cost centers)
- [ ] Defined access control requirements
- [ ] Planned bucket naming convention
- [ ] Designed security context schema
- [ ] Created custom buckets
- [ ] Configured OpenPipeline routing rules
- [ ] Configured OpenPipeline security context processors

- [ ] Verified data flow to correct buckets
- [ ] Created IAM policies
- [ ] Assigned policies to groups
- [ ] Tested access with sample users
- [ ] Documented all policies
- [ ] Created segments for team views
- [ ] Tested segment filters
- [ ] Documented segment purposes
- [ ] Planned access review process
- [ ] Configured audit logging
- [ ] Created runbook for access management

```
// Audit bucket distribution
fetch logs, from:-1h
| summarize
    recordCount = count(),
    by:{dt.system.bucket}
| sort recordCount desc
```

Exercise 2: Complete Architecture Example

```
// Audit security context coverage
fetch logs, from:-1h
| summarize
    total = count(),
    withContext = countIf(isNotNull(dt.security_context)),
    withoutContext = countIf(isNull(dt.security_context))
| fieldsAdd coverage = round(toDouble(withContext) / toDouble(total) * 100,
decimals: 2)
```

Exercise 3: Complete Architecture Example

```
// Audit bucket retention settings
fetch dt.system.buckets
| fields name, dt.system.table, retention_days
| sort dt.system.table asc, retention_days desc
```

Lab Summary

You have completed 3 hands-on exercises for **ORGNZ-09: Enterprise Data Organization Patterns**.

Exercises Completed

- [ ] Exercise 1: Complete Architecture Example
- [ ] Exercise 2: Complete Architecture Example
- [ ] Exercise 3: Complete Architecture Example

Next Steps

Continue with **ORGNZ-10** for the next notebook.
